

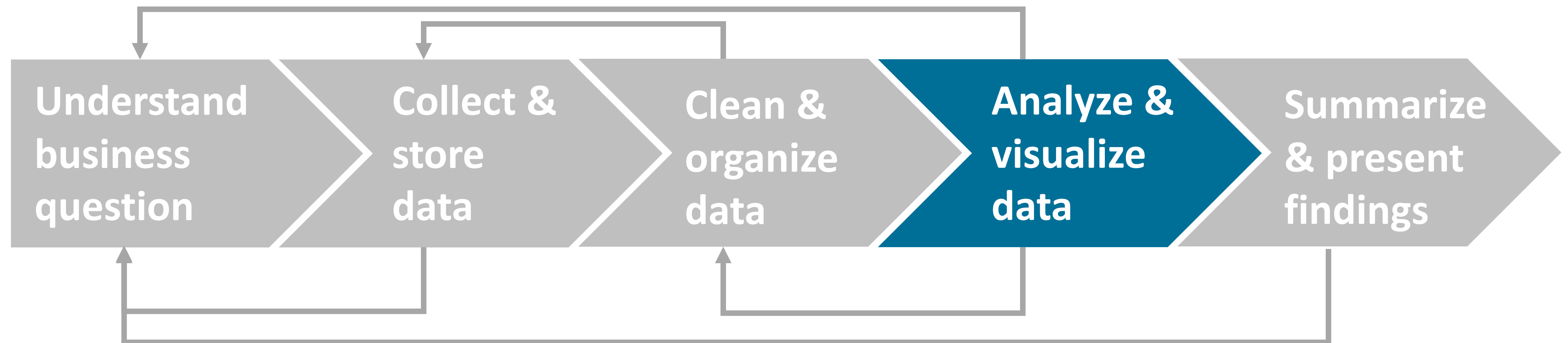
CIS 5357

Computing for Data Analytics

Week 11

Last week

- Exploring data with Python's pandas library
- Visualizing data in Python
- Coding assignment



Quick review

Calculate descriptive statistics for numeric values

`.describe(), .sum(), mean(), min(), max(), median(), ...`

Evaluate descriptive statistics for non-numeric values

`.describe(include=["object", "boolean"])`

Look for entries within a list of values

`isin(), unique()`

Get a frequency count of values

`.value_counts()`

Determine the percentage of records by category

`.value_counts(normalize = True)`

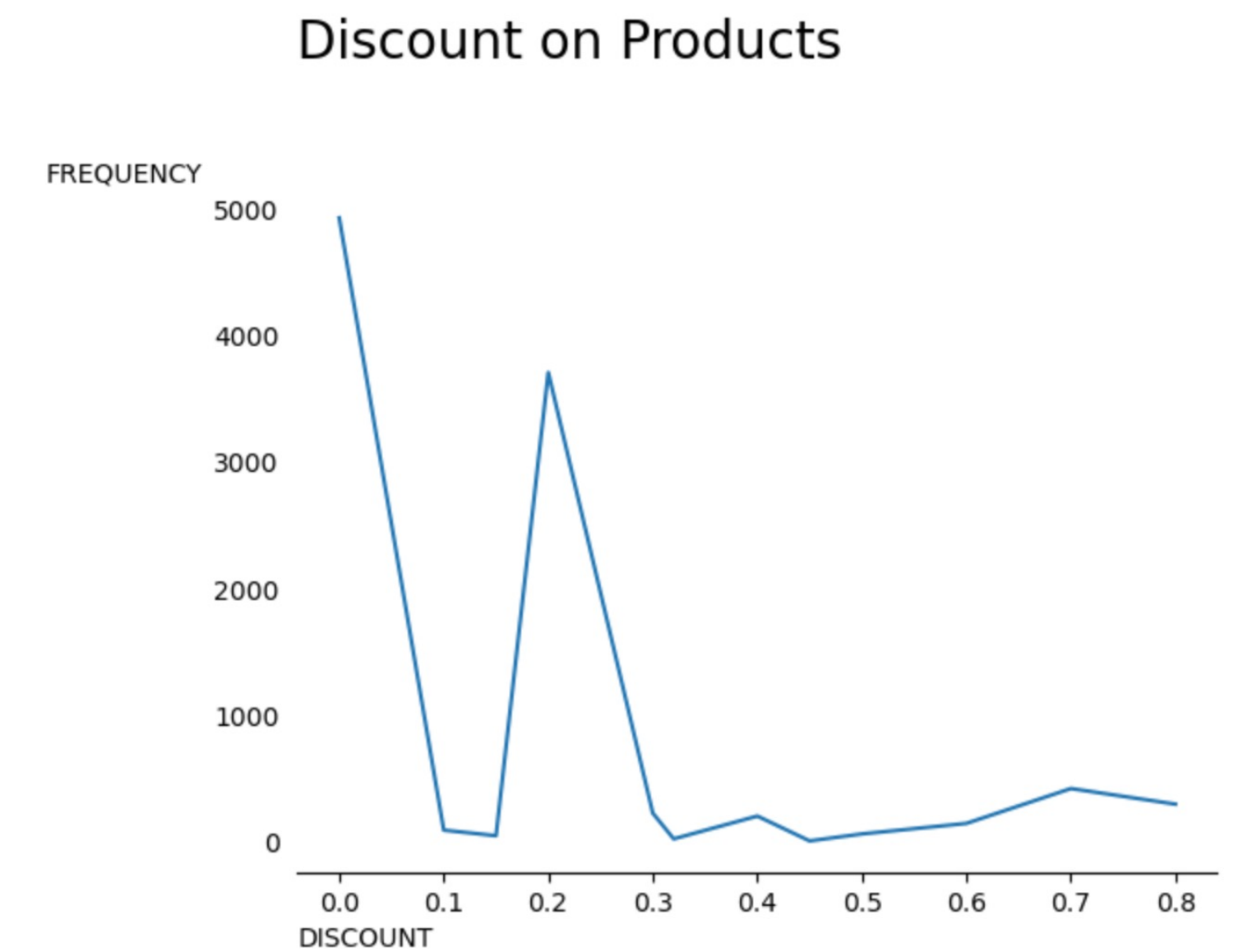
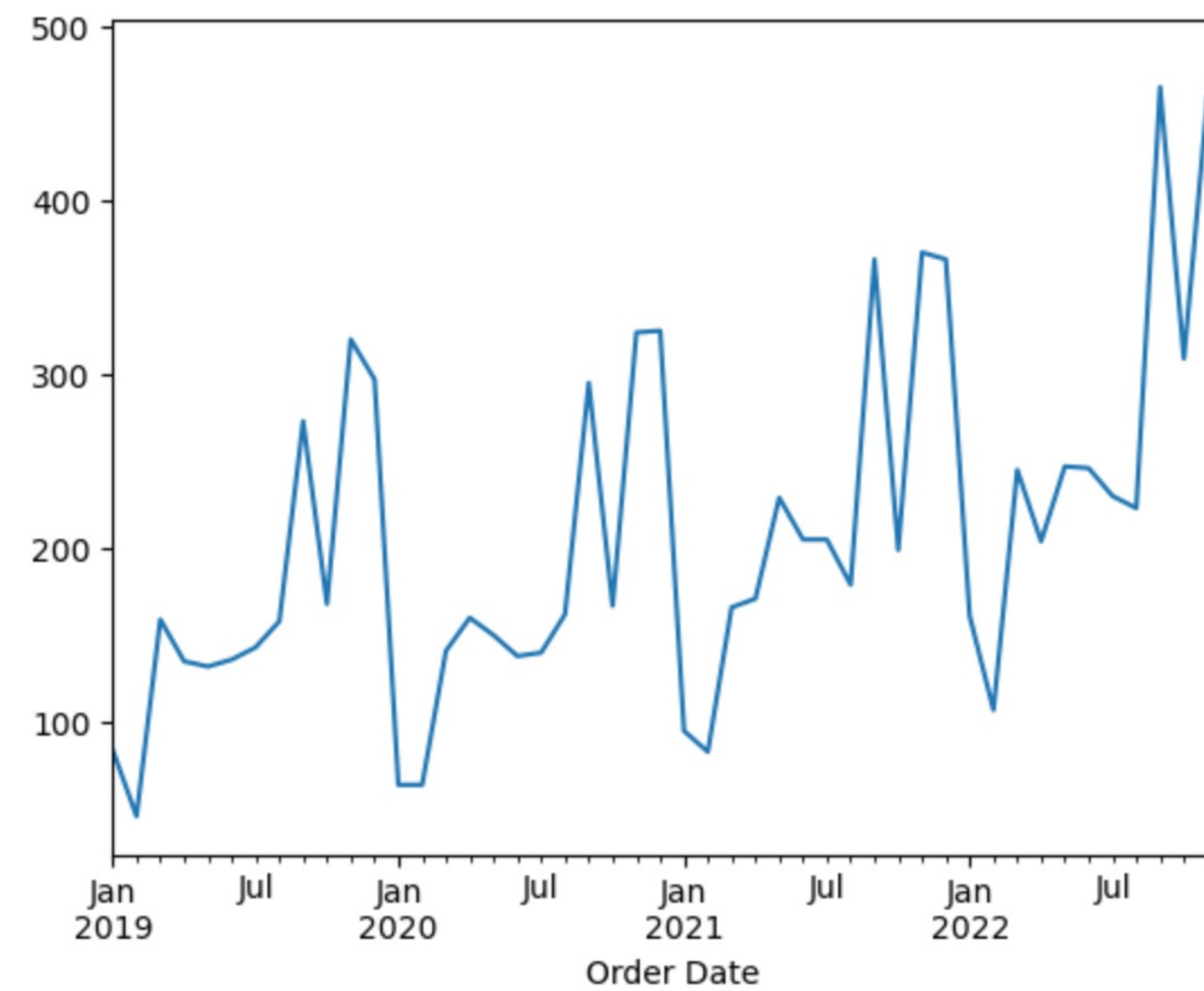
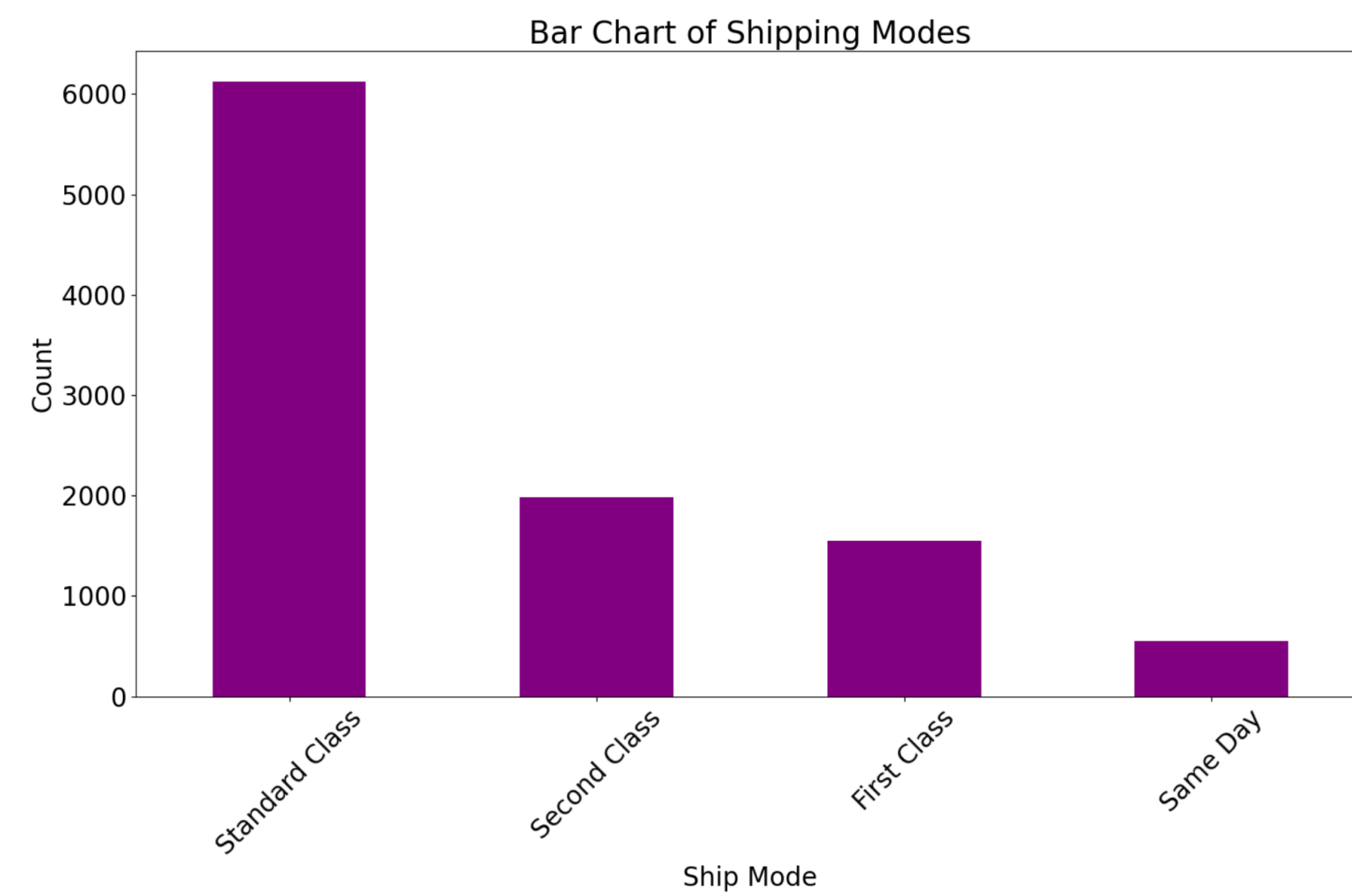
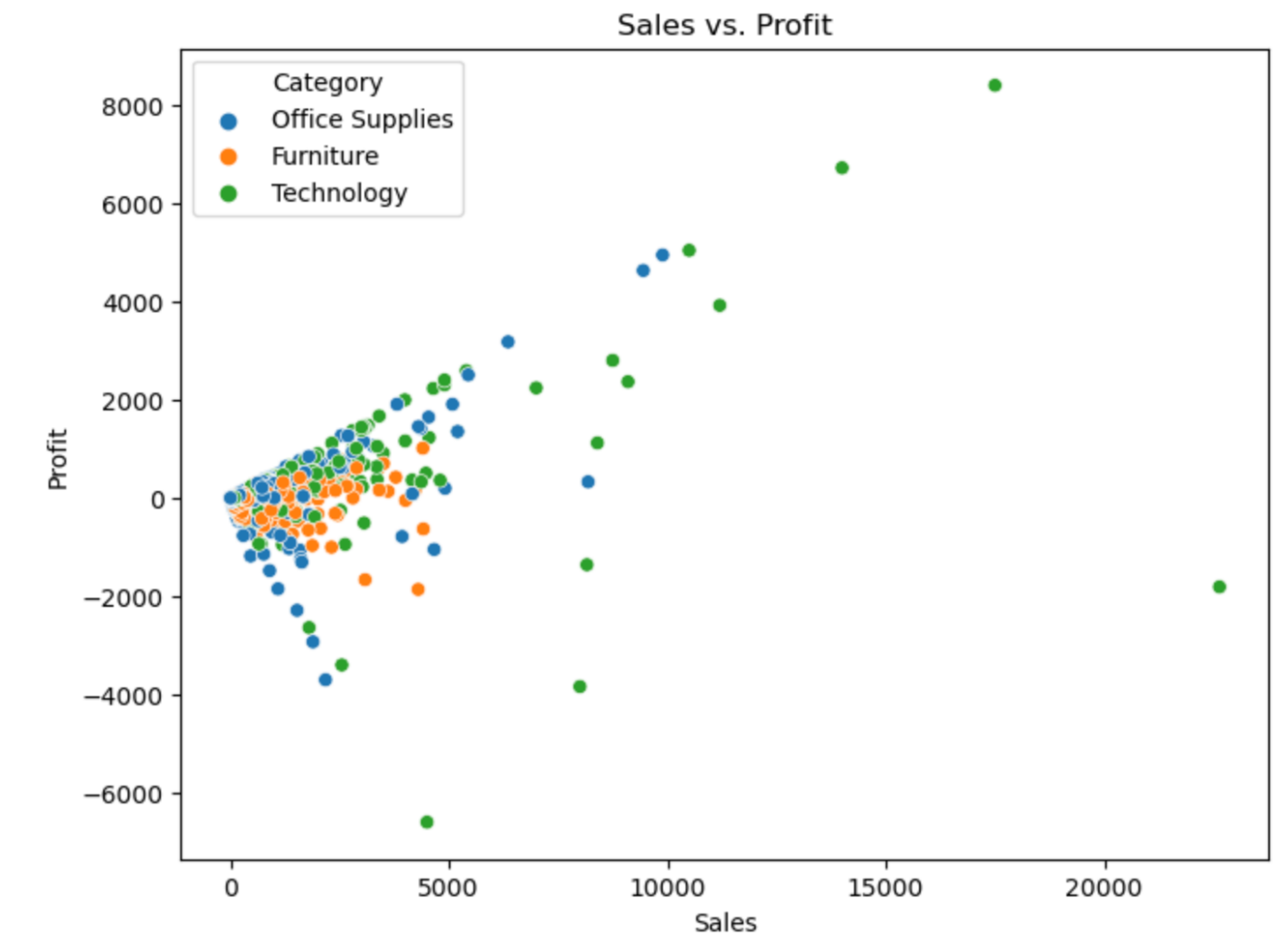
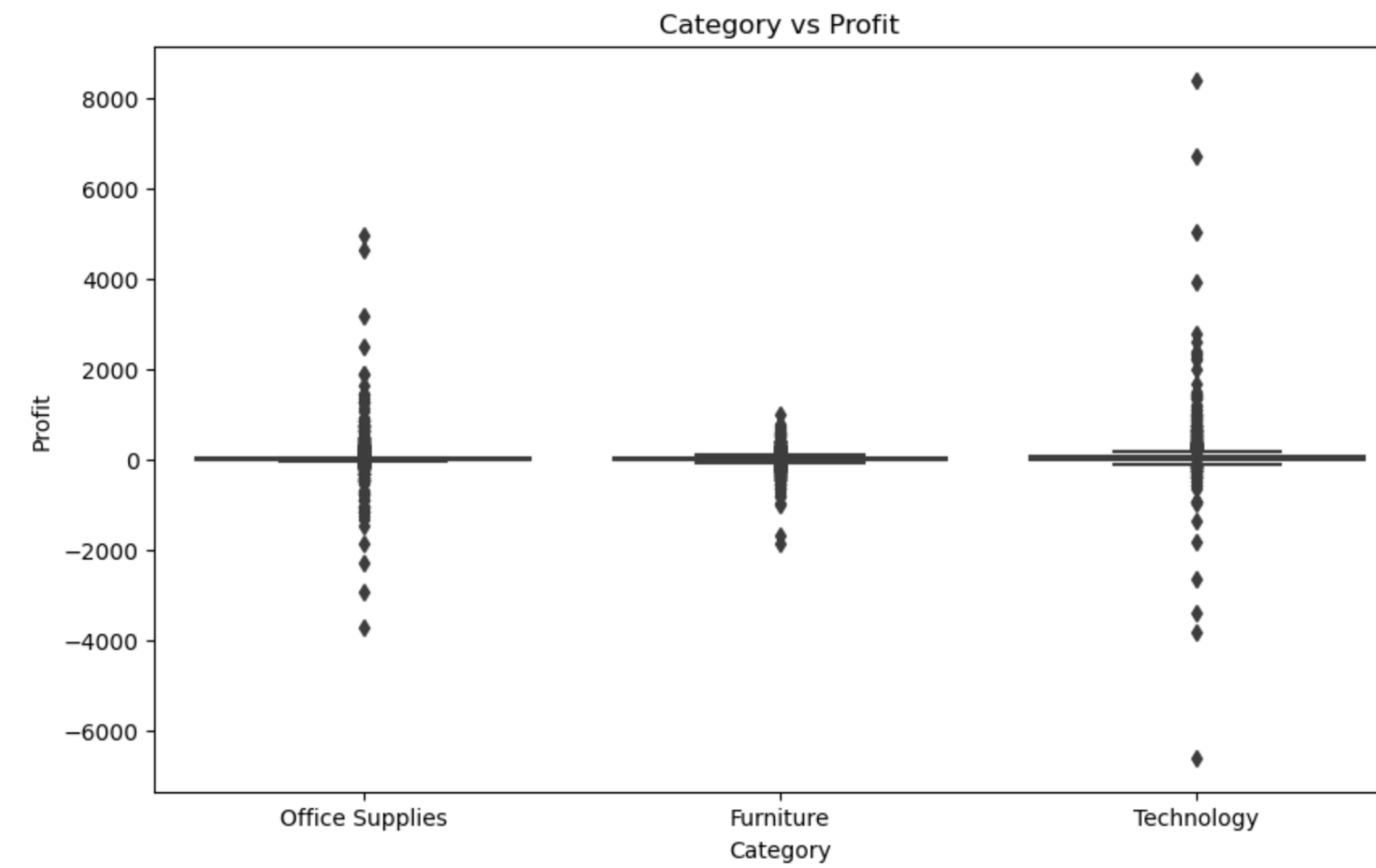
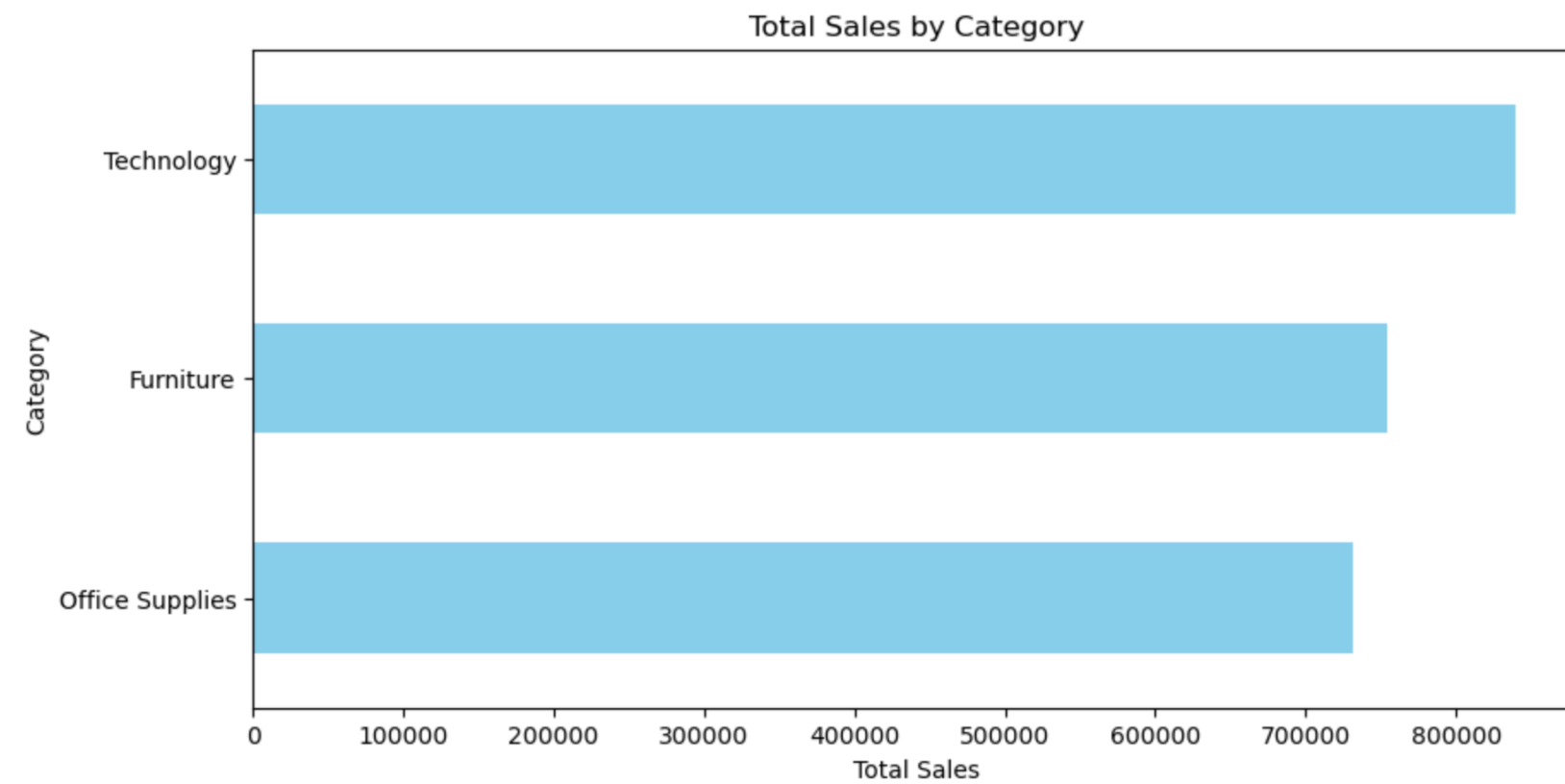
Various ways to filter records

`conditional statements, numpy where statements`

Matplotlib function to visualize data

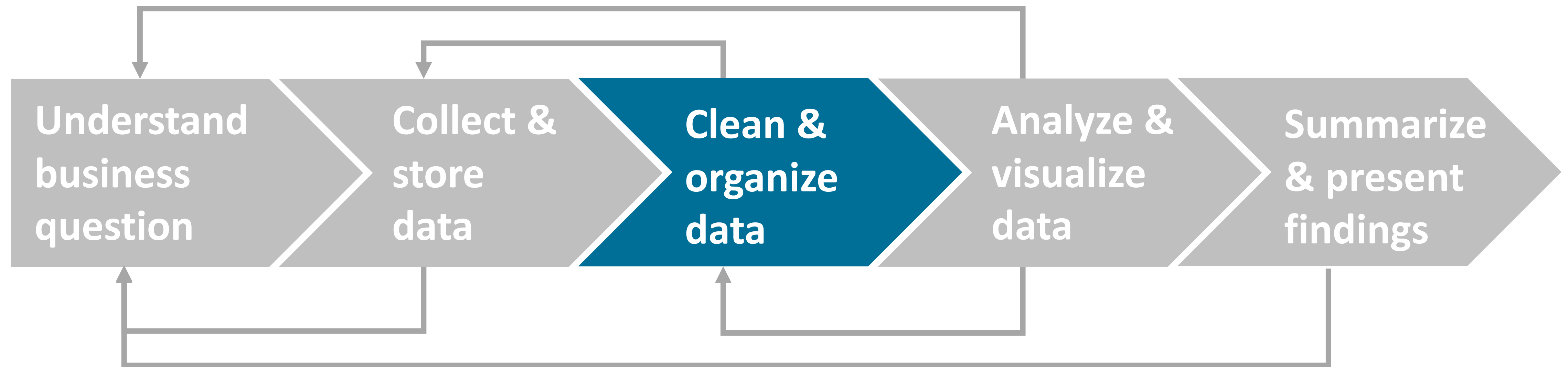
`plot(kind = "chart type")`

Assignment



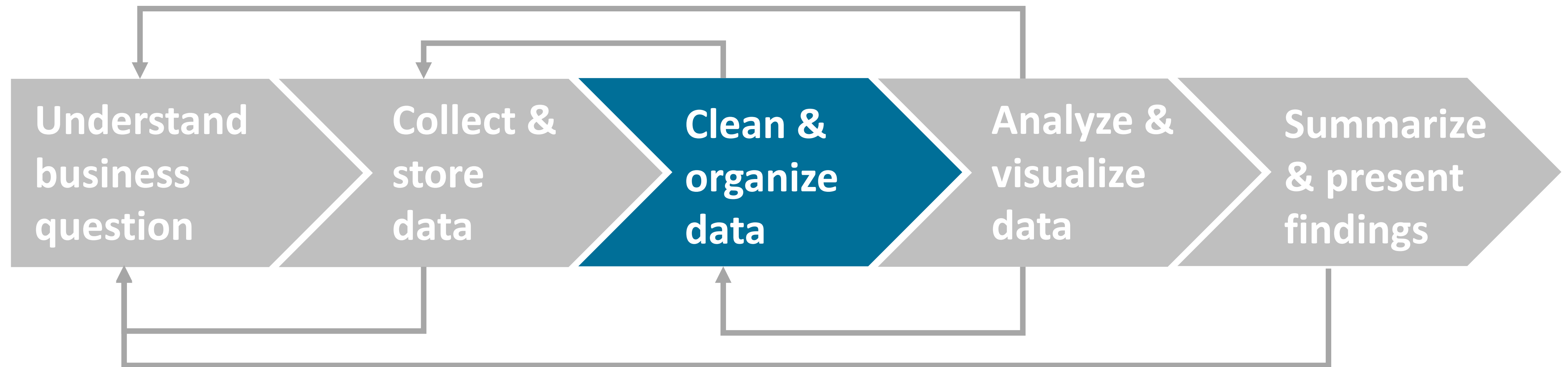
This week

- editing DataFrames in pandas
- approaches to address inaccurate and missing data



This week

- **editing DataFrames in pandas**
- approaches to address inaccurate and missing data



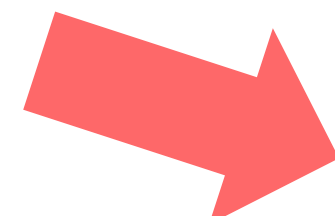
Download code to follow along

Module 11

Read and View

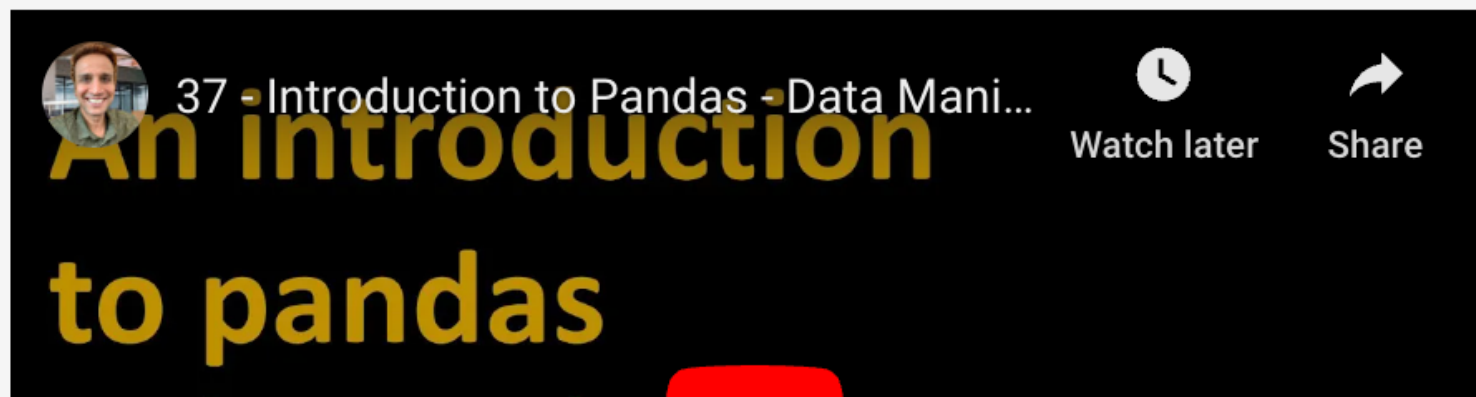
Read

- Textbook chapter 6 pages 217-220, 237-251
- [Automate Basic Data Cleaning Using Python](#) ↓
- [Pandas dataframe documentation](#) ↗
- [Pandas loc\(\) function](#) ↗
- [Practice code](#) ↓ for lecture



View

[Introduction to Pandas - Data Manipulation](#) ↗



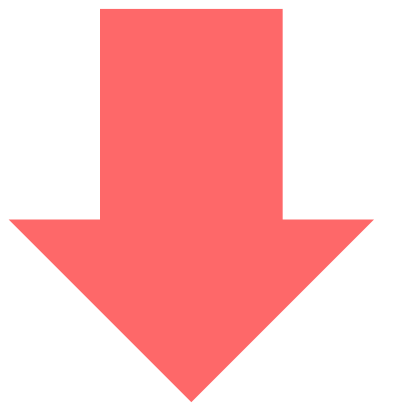
Editing data frames in pandas

- Add columns
- Remove columns
- Add rows
- Remove rows
- Edit columns
- Edit specific values

Add a column

We simply refer to the column on the left-hand-side and set it equal to the series we want it to contain. This is similar to creating a new key/value pair in a dictionary.

```
df_orders['HAS_DRINK'] = (df_orders.DRINKS > 0)
```



	ORDER_ID	DATETIME	RESTAURANT_ID	TYPE	DRINKS	COOKIES	MAIN	BASE	SIDE_1	SIDE_2	HAS_DRINK
0	O1820060	2018-10-11 17:25:50	R10002	IN_STORE	1.0	2.0	NaN	NaN	NaN	NaN	True
1	O1011112	2018-05-31 11:35:00	R10003	IN_STORE	0.0	0.0	NaN	NaN	NaN	NaN	False

Note this only works using the square-bracket notation—if you were to try to add a column using the dot notation, for example using `df_orders.HAS_DRINK = (df_orders.DRINKS > 0)`, the column would not be added

Remove a column

Use the **drop()** function with the name of the column to remove the specified data.

```
df_orders.drop(columns='HAS_DRINK', inplace=True)
```

- You must use the argument `inplace = True` to update the DataFrame
- If you want to drop multiple columns, you can pass a list to the `columns` argument of the function instead of a string.

Add rows

Use the **concat()** function to add rows of data.

```
another_new_restaurant = ({'RESTAURANT_ID': ['R00000'],  
    'NAME': ['Famous Spot'],  
    'ADDRESS': ['12345 First St, Anytown, NY 11000'],  
    'LAT': [40.2] ,  
    'LONG': [70.1],  
    'OPENING_DATE': ['3/28/2024'],  
    'DELIVERY_START': ['2024-03-28']})  
df_new_restaurant2 = pd.DataFrame(another_new_restaurant)  
df_restaurants = pd.concat([df_restaurants, df_new_restaurant2])
```

Remove rows based on conditions

Can also use the **drop()** function to remove rows of data

```
df_restaurants.drop(df_restaurants  
                    [df_restaurants['RESTAURANT_ID'].isin(['R00000', 'R10000'])]  
                    .index  
                    , inplace=True)
```

1. filter the records to drop

2. To drop rows you need to include the index value for the records you wish to drop

3. inplace = True does the operation in place; otherwise it returns a copy.

Edit a column

We simply “overwrite” the existing column with its new value.

```
df_orders['DRINKS'] = df_orders.DRINKS * 2
```

OR

```
df_orders.DRINKS = df_orders.DRINKS * 2
```

Note that when editing a column (rather than adding one), the dot notation does work on the left-hand side of the equal sign.

```
# do the opposite calculation to reverse the change
```

```
df_orders['DRINKS'] = df_orders.DRINKS / 2
```

Editing specific values

We can use the **loc** keyword to **filter and change a DataFrame at the same time**

For example, we create a new column in the summarized orders table to categorize order volume. To begin we'll set everything to high.

```
df_summarized_orders['ORDER_VOLUME'] = 'HIGH'
```

Then we can modify the values using *loc*

```
df_summarized_orders.loc[df_summarized_orders.NUM_ORDERS <= 1200,  
    'ORDER_VOLUME'] = 'MEDIUM'
```

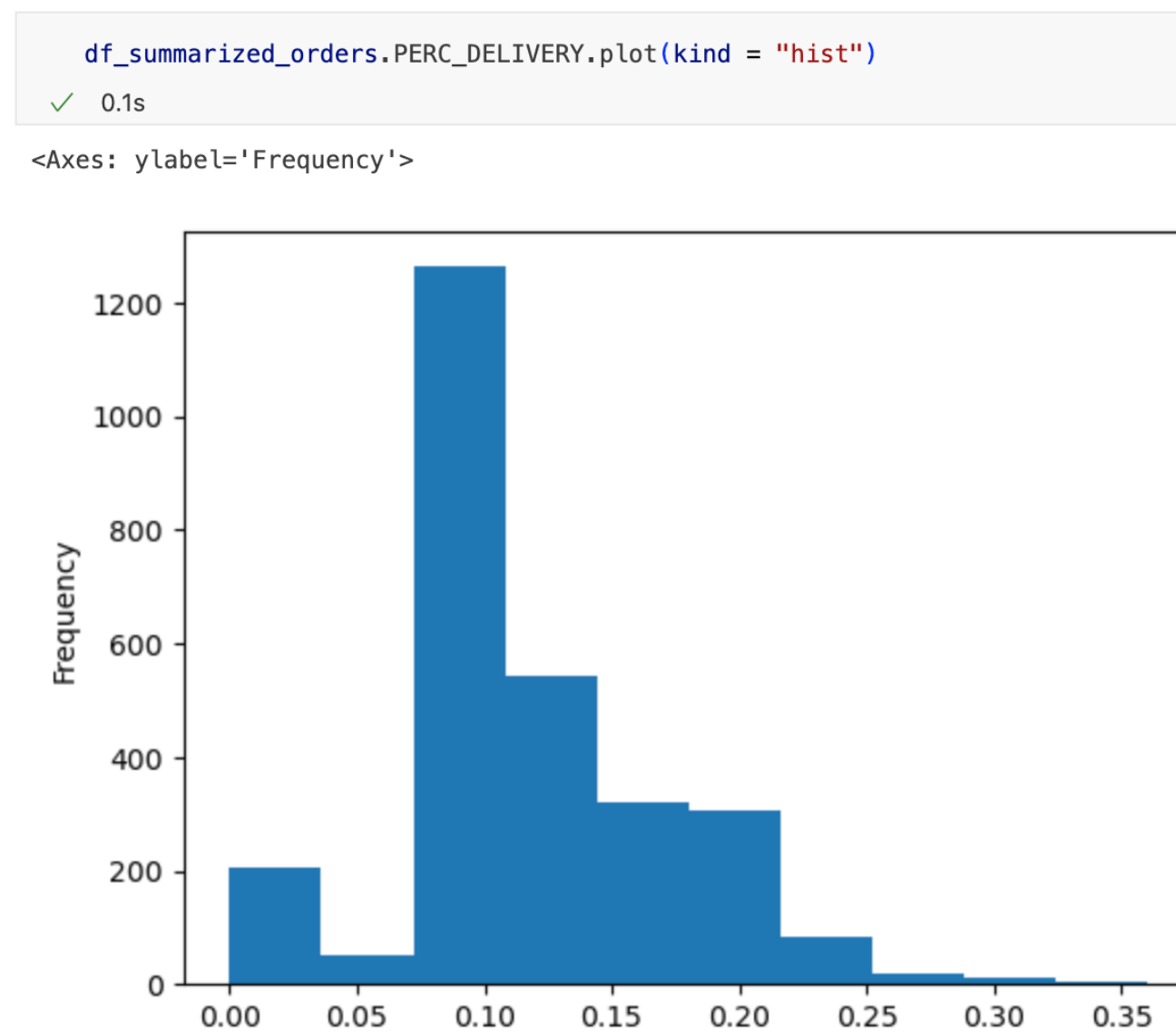
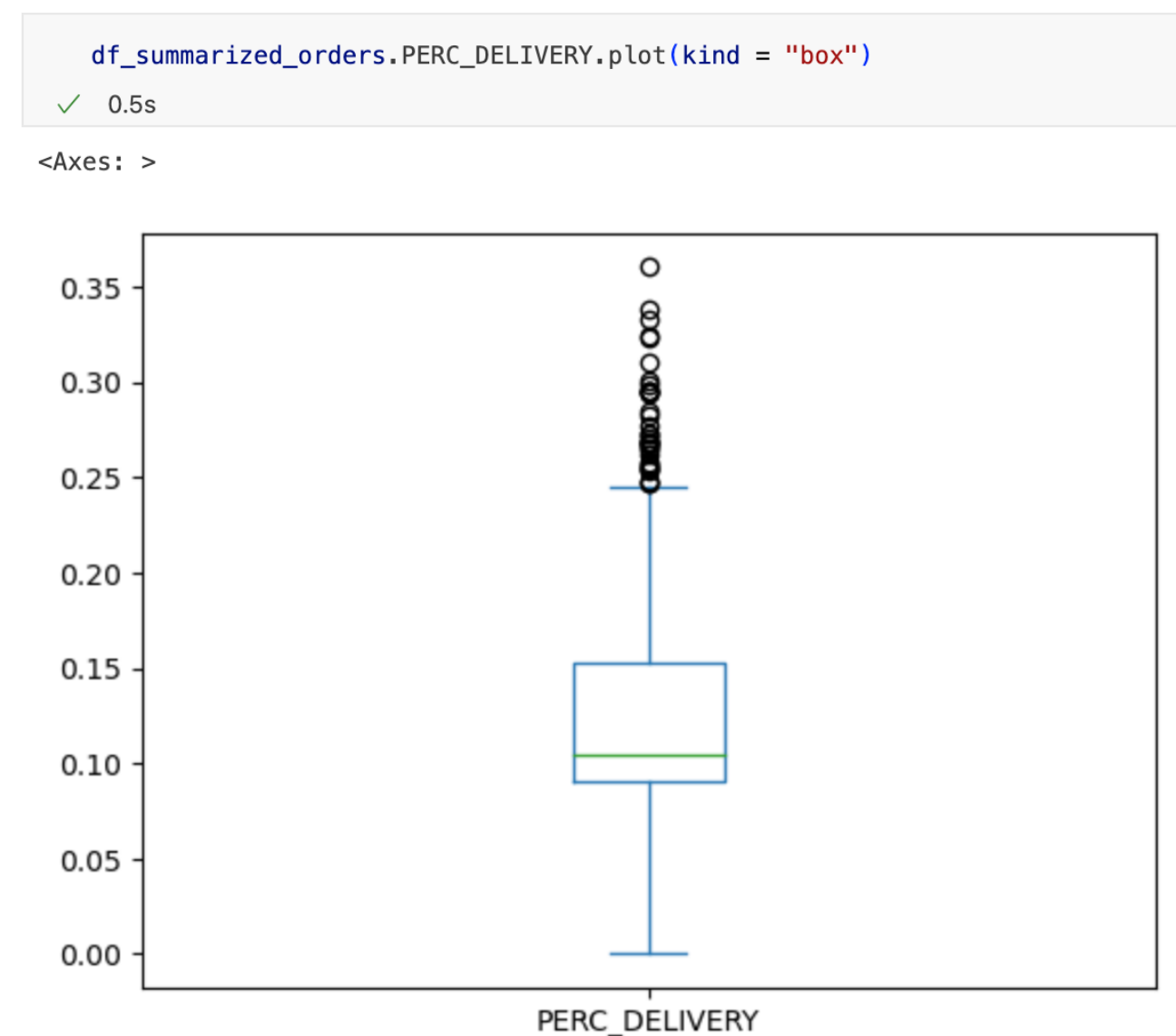
```
df_summarized_orders.loc[df_summarized_orders.NUM_ORDERS <= 600,  
    'ORDER_VOLUME'] = 'LOW'
```

Challenge 1: delivery volume

Look at the distribution of PERC_DELIVERY in the *df_summarized_orders* table, then create a new column called DELIVERY_VOLUME that assigns each row to a category (none, low, medium, high) based on the value of PERC_DELIVERY.

One possible solution

Look at the distribution of PERC_DELIVERY in the df_summarized_orders table, then create a new column called DELIVERY_VOLUME that assigns each row to a category (none, low, medium, high) based on the value of PERC_DELIVERY.



```
df_summarized_orders.PERC_DELIVERY.describe()
```

✓ 0.0s

count	2806.000000
mean	0.116700
std	0.055078
min	0.000000
25%	0.090095
50%	0.104963
75%	0.152853
max	0.360215

Name: PERC_DELIVERY, dtype: float64

None: 0%

Low: >0%<=10%

Medium: >10%<=15%

High: >15%

One possible solution

Look at the distribution of PERC_DELIVERY in the df_summarized_orders table, then **create a new column called DELIVERY_VOLUME that assigns each row to a category (none, low, medium, high)** based on the value of PERC_DELIVERY.

```
df_summarized_orders['DELIVERY_VOLUME'] = 'NONE: 0%'
df_summarized_orders.loc[df_summarized_orders.PERC_DELIVERY > 0, 'DELIVERY_VOLUME'] = 'LOW: >0%<=10%'
df_summarized_orders.loc[df_summarized_orders.PERC_DELIVERY > 0.1, 'DELIVERY_VOLUME'] = 'MEDIUM: >10%<=15%'
df_summarized_orders.loc[df_summarized_orders.PERC_DELIVERY > 0.15, 'DELIVERY_VOLUME'] = 'HIGH: >15%'
df_summarized_orders.DELIVERY_VOLUME.value_counts()
```

✓ 0.0s

```
LOW: >0%<=10%      984
MEDIUM: >10%<=15%  904
HIGH: >15%         712
NONE: 0%           206
Name: DELIVERY_VOLUME, dtype: int64
```

Can also utilize the `apply()` function

The **`apply()`** function can take any function you define and apply it sequentially, row by row, to an entire series.

```
def delivery_level(percent_delivery):  
    if percent_delivery == 0:  
        return 'None'  
    elif percent_delivery <= .10:  
        return 'Low'  
    elif percent_delivery <= .15:  
        return 'Medium'  
    else:  
        return 'High'  
  
df_summarized_orders['DELIVERY_LEVEL'] =  
    df_summarized_orders.PERC_DELIVERY.apply(delivery_level)
```

For simple cases use lambda

Lambda is a nameless, anonymous function that is useful for simple expressions, usually a single line of a statement.

```
df_summarized_orders['ANY_DELIVERIES'] =  
    df_summarized_orders.PERC_DELIVERY  
    .apply(lambda perc_delivery: perc_delivery > 0)
```

1. Lambda tells Python we're about to define a one line function

2. The name of the argument comes next followed by a colon

3. Then the expression we want to return

Map & replace also modify values

The map method can be used to **replace values in a column** by passing a dictionary of the form **{old_value: new_value}** as its argument:

```
d = {False: "No", True: "Yes"}  
df_summarized_orders["ANY_DELIVERIES_2"] =  
    df_summarized_orders["ANY_DELIVERIES"].map(d)
```

The same thing can be done with the replace method:

```
df_summarized_orders =  
    df_summarized_orders.replace({"ANY_DELIVERIES": d})
```

Challenge 2: order types

Create a new column in `df_orders` that uses numeric values to indicate the order type. Rows for in store orders should be 1, pickup orders should be 2 and delivery orders should be 3.

Challenge 2: order types

Create a new column in `df_orders` that uses numeric values to indicate the order type. Rows for in store orders should be 1, pickup orders should be 2 and delivery orders should be 3.

```
# OPTION 1: use apply
def order_type(type):
    if type == "IN_STORE": return 1
    elif type == "DELIVERY": return 2
    else: return 3

df_orders['TYPE_CD'] = df_orders.TYPE.apply(order_type)
```

Challenge 2: order types

Create a new column in `df_orders` that uses numeric values to indicate the order type. Rows for in store orders should be 1, pickup orders should be 2 and delivery orders should be 3.

```
# OPTION 2: use map
```

```
type_cd= {"IN_STORE": 1, "DELIVERY": 2, "PICKUP": 3}
```

```
df_orders["TYPE_CD_2"] = df_orders["TYPE"].map(type_cd)
```

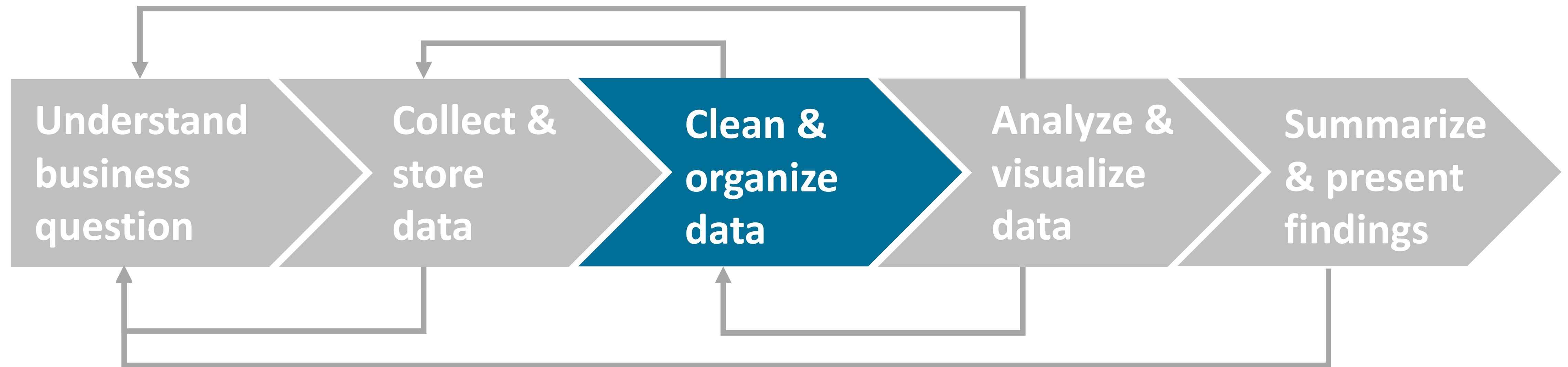
```
# OPTION 3: use replace
```

```
type_cd= {"IN_STORE": 1, "DELIVERY": 2, "PICKUP": 3}
```

```
df_orders = df_orders.replace({"TYPE": type_cd})
```

This week

- editing DataFrames in pandas
- **approaches to address inaccurate and missing data**



Missing values

Missing data is defined as the values or data that is not stored (or not present) for some variable(s) in the given dataset

- Pandas denotes missing values a NaN

Missing values occur for various reasons

- Corrupted data
- Failure in recording the values due to human error
- The user has not provided the values intentionally
- Observations not recorded for some reasons

What to do with missing values?

If missing values will bias analysis results they should be addressed

Delete entries with missing values

- Quick and easy to drop a row or column of data
- But this may end up deleting some useful data from the dataset

Impute data for the missing values

- Replace with mean (assuming no outliers)
- Replace with median (if data is skewed)
- Replace with mode (when categorical data)
- Replace with arbitrary value (e.g., 0)

Find missing values

The **isnull()** function returns a series that is as long as the original one. It contains True whenever the corresponding value is missing, and False otherwise.

```
df_orders.MAIN.isnull()
```

Note the **notnull()** function does the opposite—returning True when the value is populated and False when it is missing

Orders that do not include a bowl

```
df_orders.MAIN.isnull().value_counts()
```

✓ 0.0s

```
False    2275639  
True      111585  
Name: MAIN, dtype: int64
```

```
df_orders.MAIN.isnull().value_counts(normalize = True)
```

✓ 0.0s

```
False    0.953257  
True     0.046743  
Name: MAIN, dtype: float64
```

Fill in missing values

Pandas function **fillna()** allows us to fill in missing values

```
df_orders.MAIN.fillna(0)
```

✓ 0.0s

0	0
1	0
2	I0
3	I0
4	I1
	..
2387219	I3
2387220	I1
2387221	I2

Ways to discover anomalies

- Eyeball, sort, or filter the dataset
- Generate descriptive statistics
- Plot data: histograms or boxplots

What is an outlier?

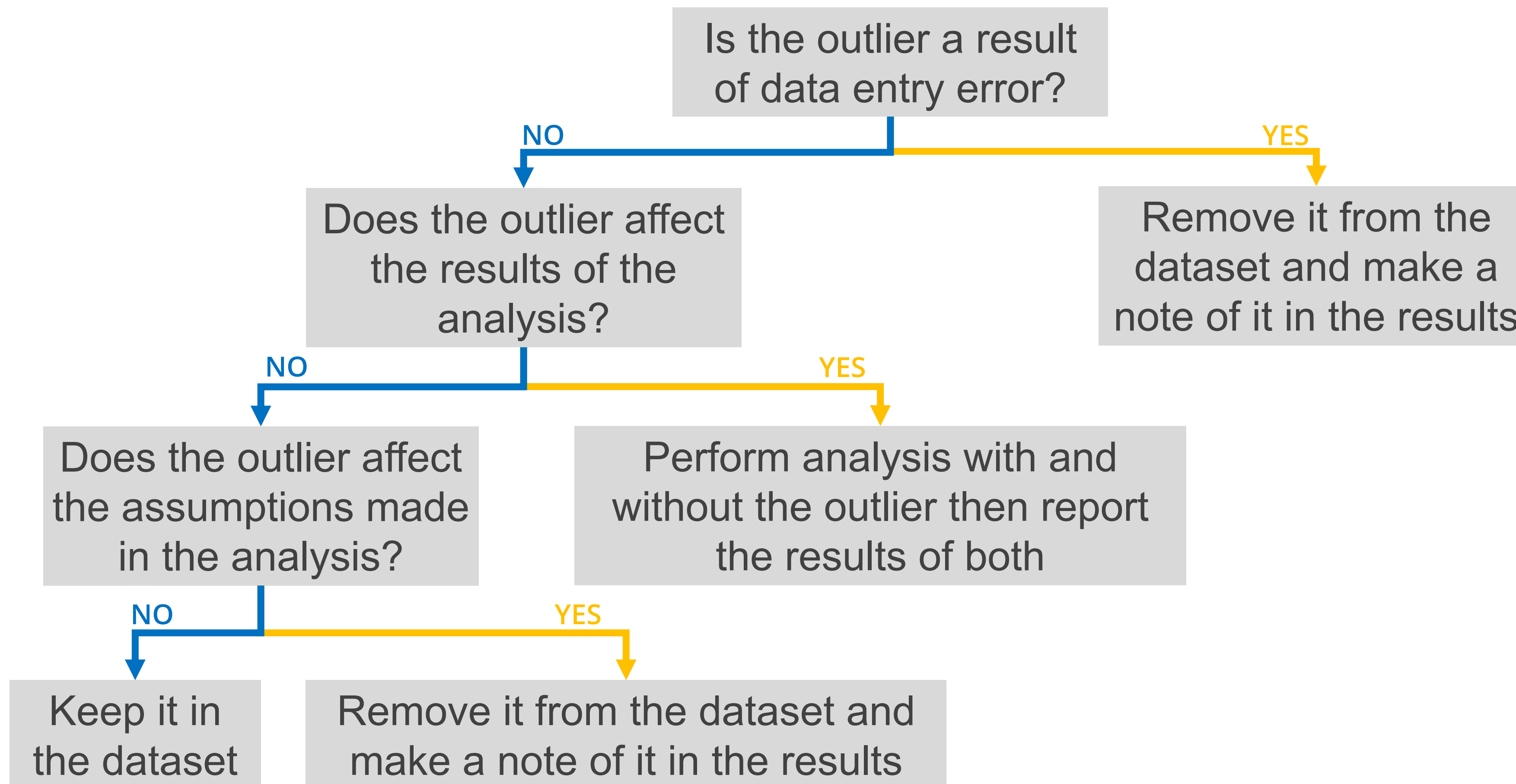
An outlier is an observation that lies *abnormally* far away from other values in a dataset

- values that lie more than 3 standard deviations from the mean
- values that lie beyond 1.5 times the interquartile range

What to do about outliers?

- Outliers can be problematic because they can affect the results of analysis
- But....outliers can also be informative about the data you're studying because they can reveal abnormal cases or individuals that have rare traits

Decide to remove or keep outliers



Challenge 3: abnormal orders

Explore the Dig data to check whether the number of orders are abnormally high or low on any given day at a given restaurant. If there are any outliers, consider what to do about them.

Challenge 3: abnormal orders

Explore the Dig data to check whether the number of orders are abnormally high or low on any given day at a given restaurant. If there are any outliers, consider what to do about them.

```
df_summarized_orders.NUM_ORDERS.isnull().value_counts()
```

✓ 0.0s

```
False    2806  
Name: NUM_ORDERS, dtype: int64
```

Challenge 3: abnormal orders

Explore the Dig data to check whether the number of orders are abnormally high or low on any given day at a given restaurant. If there are any outliers, consider what to do about them.

```
df_summarized_orders.describe()
```

✓ 0.0s

	NUM_ORDERS	PERC_DELIVERY
count	2806.000000	2806.000000
mean	850.756949	0.116700
std	195.490367	0.055078
min	200.000000	0.000000
25%	739.000000	0.090095
50%	833.000000	0.104963
75%	949.000000	0.152853
max	1396.000000	0.360215

$$850.76 \pm 3 \times 195.49 =$$
$$850.76 \pm 586.47 =$$
$$264.29 \text{ to } 1,437.23$$

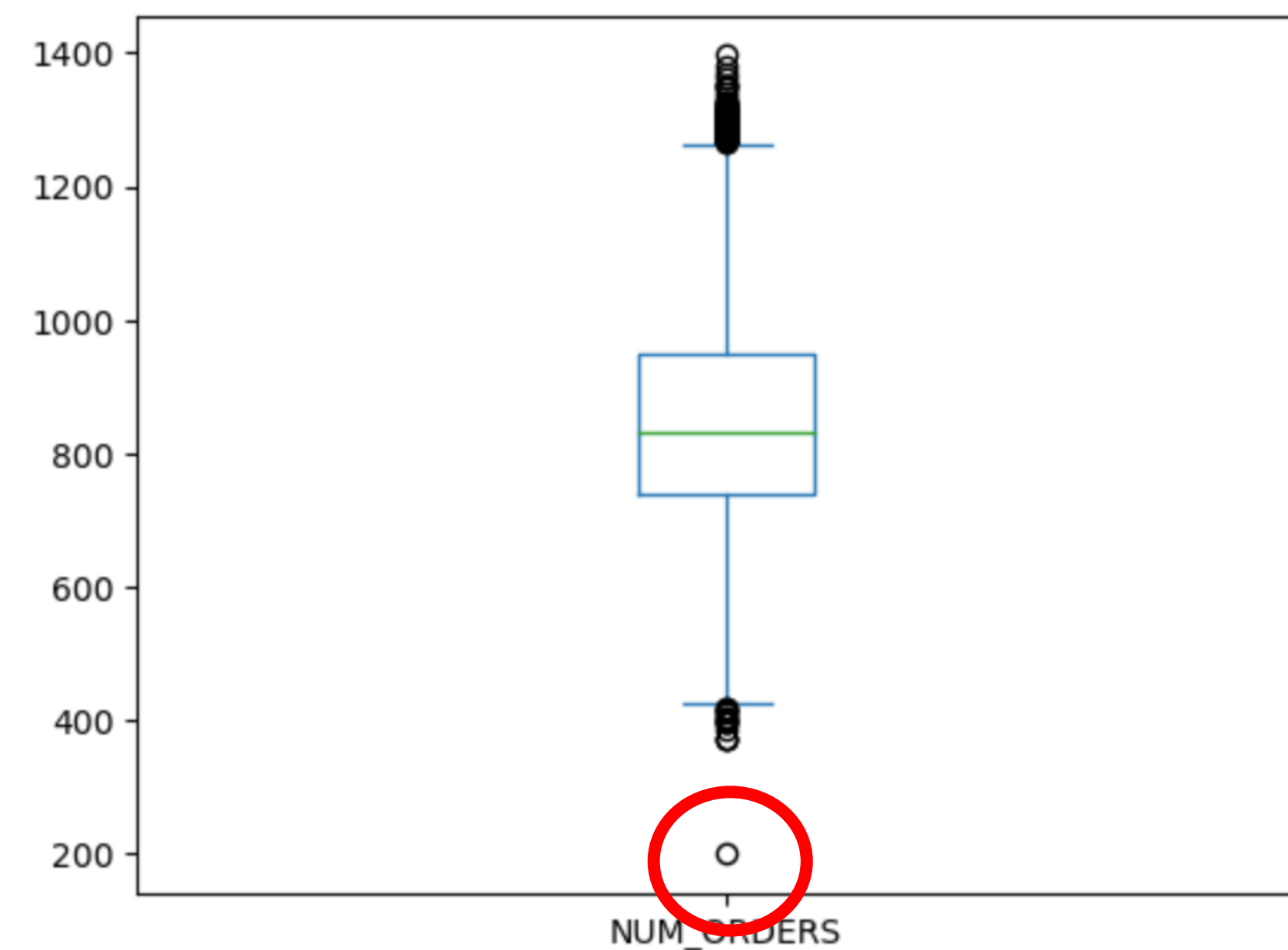
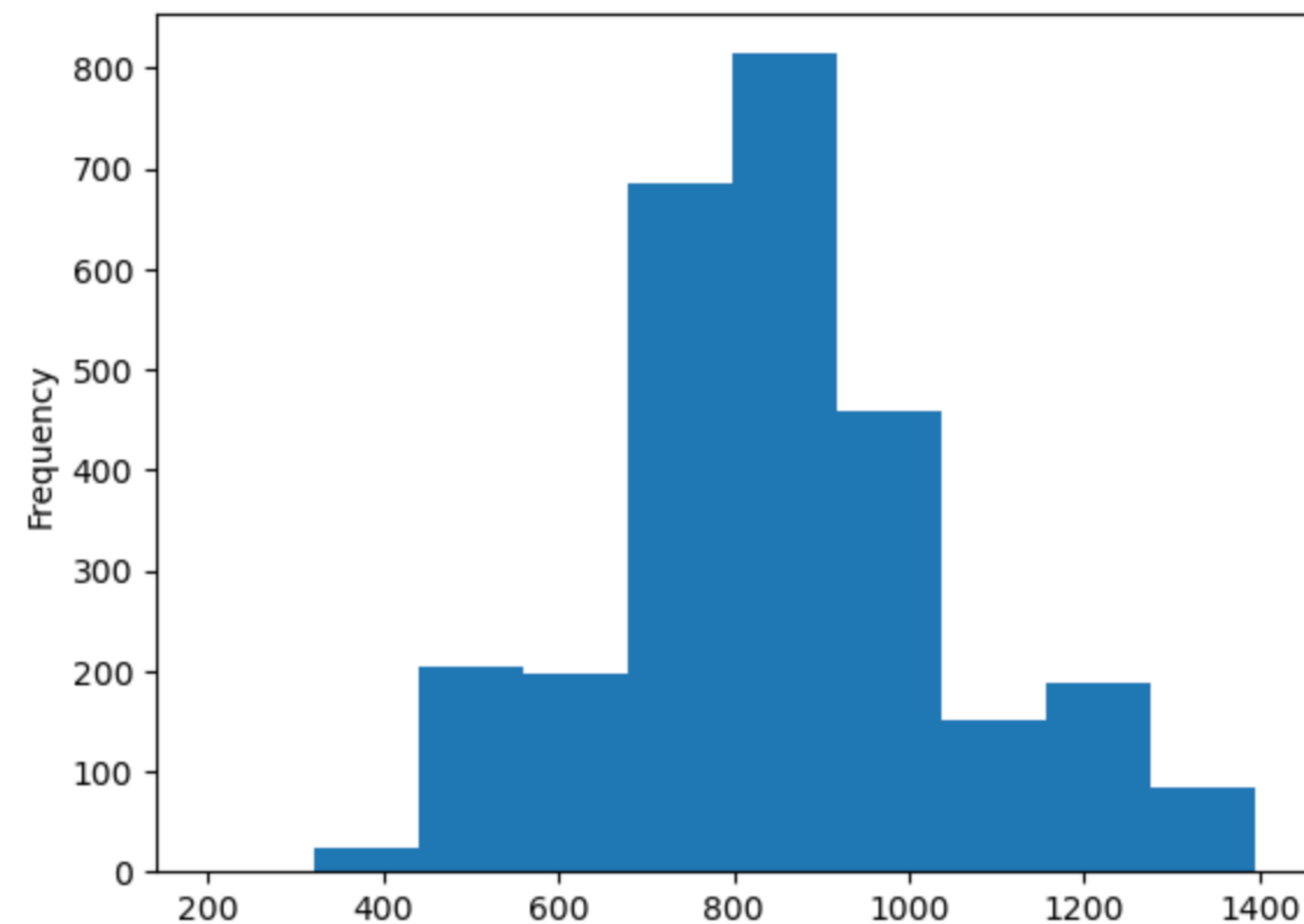
```
df_summarized_orders.loc[df_summarized_orders.NUM_ORDERS <= 264]
```

✓ 0.0s

	RESTAURANT_NAME	DATE	NUM_ORDERS	PERC_DELIVERY
1066	Midtown	2018-03-17	200	0.0

Challenge 3: abnormal orders

Explore the Dig data to check whether the number of orders are abnormally high or low on any given day at a given restaurant. If there are any outliers, consider what to do about them.



Recap

Editing DataFrames in pandas

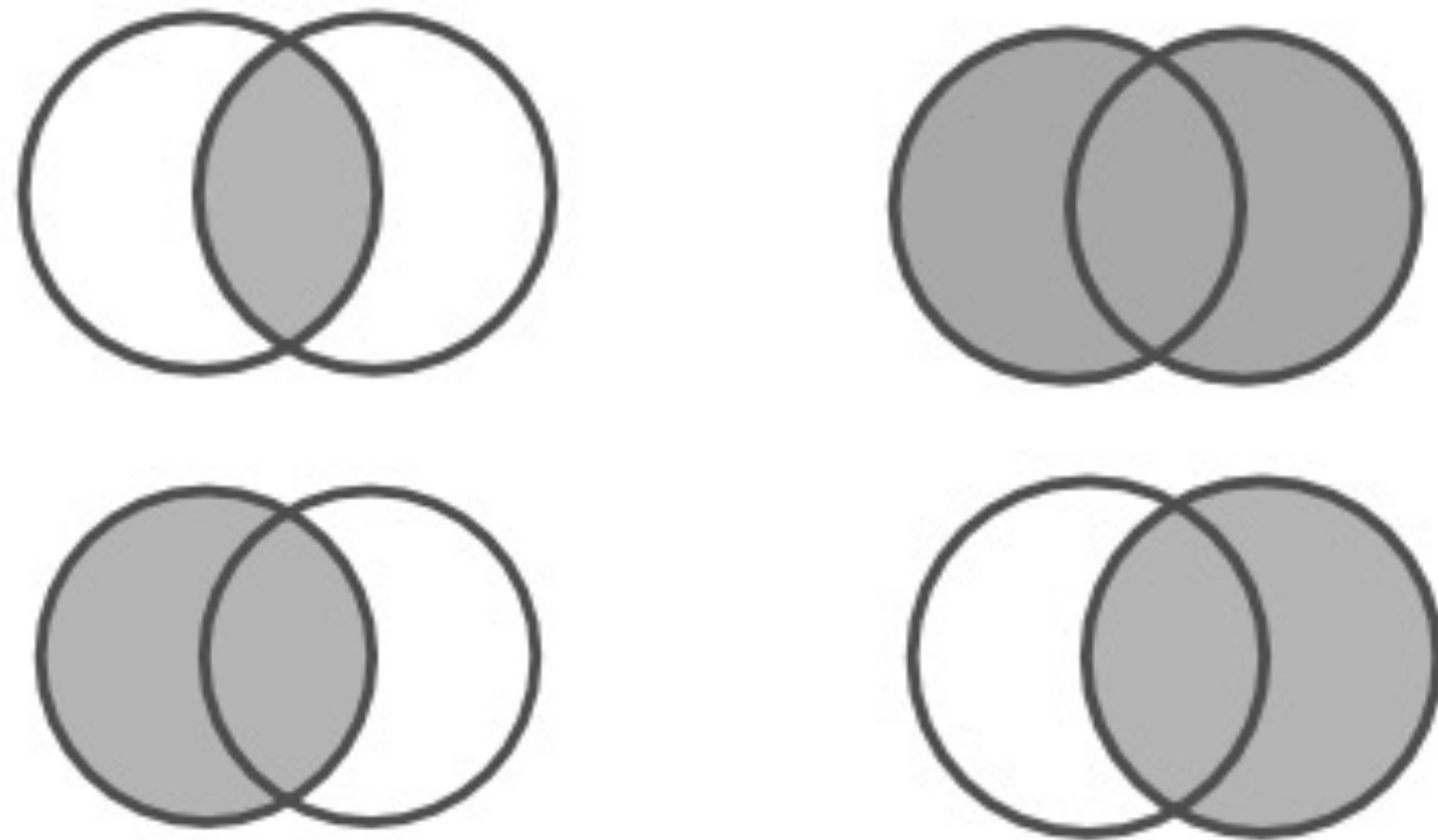
- Add columns with square brackets *df_name["new_column"]*
- Add rows with *concat()*
- Remove columns or rows with *drop()*
- Edit columns by overwriting
- Use *loc()*, *apply()*, *map()* or *replace()* to edit specific values

Address messy data

- Missing data: *isnull()*; *fillna()*
- Evaluate outliers

Next week

Next week: **bringing together data sets**



We'll meet **on Zoom**